

Nome e cognome _____ Matricola _____

La durata della prova è di 2 ore. Scrivere in stampatello maiuscolo. E' obbligatorio restituire il testo e tutti i fogli forniti, anche se non si consegna il compito.

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella, e considerato che l'ordine di arrivo dei processi è P1, P2, P3, P4, P5, tutti all'istante 0.

Process	Burst Time
P1	10
P2	1
P3	2
P4	1
P5	5

- a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di scheduling SJF e RR (quanto = 3 ms). Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro.

- b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo d'attesa e come l'avete applicato per calcolare i risultati esposti)

	SJF	RR
P1		
P2		
P3		
P4		
P5		

- c) Calcolare il tempo di turnaround per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo di turnaround e come l'avete applicato per calcolare i risultati esposti)

	SJF	RR
P1		
P2		
P3		
P4		
P5		

2. Tenuto conto che:

- o il pid del processo padre è 10 e quello del figlio è 11.

Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta

```
int main()
{
    pid_t pid;
    int status, var = 5;
    printf("Sono il processo padre.\n");

    pid = fork();

    if(pid > 0)
    {
        var++;
        printf("var = %d \n", var);

        wait(&status);
    }

    printf("var = %d \n", var);
    printf("pid = %d \n", getpid());

    return 0;
}
```

3. Dato il frammento di programma che segue:

- Quanti processi e quanti thread genera il frammento di programma seguente?
- Evidenziare, se ci sono, le corse critiche. Come si possono evitare?
- Cosa stamperà il programma?

Giustificare in estrema sintesi le risposte

```
#define NUM_THREADS 4
int shared = 0;
void *func(void* param)
{
    for (int i = 0; i < 100; ++i) {
        shared += 1;
    }

    return 0;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_create(&threads[i], NULL, func, NULL);
    }

    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_join(threads[i], NULL);
    }

    printf("%d\n", shared);

    return 0;
}
```

4. Nel contesto dei metodi per evitare i deadlock cosa si intende per "stato sicuro"?

- a) Dati tre processi P_0, P_1, P_2 e tre tipi di risorsa A (5 istanze), B (4 istanze), C (2 istanze) con le seguenti matrici di allocazione, max e disponibile, verificare se il sistema è in uno stato sicuro

	Allocazione			Max			Disponibile		
	A	B	C	A	B	C	A	B	C
P_0	2	0	1	3	2	1	1	2	0
P_1	1	1	1	4	1	1			
P_2	1	1	0	5	3	1			

- b) Se P_2 richiede (0, 1, 0) cosa succede?

5. Si consideri una memoria paginata, dato un processo con una tabella delle pagine in memoria

- a) Se ogni accesso in memoria richiede 60 ns qual è il tempo per accedere ad un indirizzo della memoria paginata?
 b) Aggiungendo una TLB con hit ratio del 60% e stimando un tempo di accesso di 3 ns qual è il EAT (Effective Access Time)?
 c) Descrivere succintamente cos'è una TLB e qual è la sua funzione

6. Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:

1, 2, 0, 2, 1, 0, 1

- a) Se il processo ha 2 frame gestiti con LRU quanti page fault vengono generati
 b) Quanti page fault vengono generati con l'algoritmo OPT

7. Descrivere in maniera sintetica ma esaustiva il concetto di working set WS all'istante t con intervallo Δ $WS(t, \Delta)$ e la sua funzione.

- a) Data la stringa dei riferimenti 1, 3, 3, 7, 1, 1, 9, 10, 7 scrivere il contenuto di $WS(9,5)$

8. Assumendo 2000 cilindri e testina su 1100 (con richiesta precedente a 801)

- a) Data la coda di richieste 100, 50, 1500, 1700, 400 come vengono servite le richieste con l'algoritmo C-LOOK
 b) Qual è la distanza in cilindri percorsi della testina con l'algoritmo C-LOOK

9. Domande (più di una risposta può essere corretta)

- a) **Quale delle seguenti affermazioni sul copy_on_write è vera**
 a. È una tecnica che per la sincronizzazione tra processi evitando corse critiche
 b. È un metodo che permette al processo figlio di condividere inizialmente le stesse pagine del processo padre
 c. È un metodo per velocizzare la scrittura sul file system
 d. È una tecnica che consente di minimizzare il numero di pagine allocate per un nuovo processo
- b) **Quale tra queste affermazioni su thread e processi è vera**
 a. Due thread possono condividere lo stesso PID e PPID
 b. Un thread può esistere all'interno di più processi
 c. Quando all'interno di un processo viene creato un thread, questo riceve una copia dello spazio di indirizzamento del processo che lo ha creato
 d. Due thread di due processi diversi possono determinare una corsa critica su una variabile globale
- c) **Quale tra queste affermazioni sull'allocazione dei file è vera**
 a. L'allocazione linkata di un file presenta problemi di frammentazione esterna

- b. L'allocazione contigua soffre problemi di frammentazione esterna
 - c. L'allocazione indicizzata presenta problemi di frammentazione esterna ma risolve il problema della frammentazione interna
 - d. L'allocazione indicizzata favorisce l'accesso random al file
- d) **Quale tra queste affermazioni sui meccanismi di sincronizzazione è vera**
- a. Lo spin-lock determina un context switch perché sospende un processo
 - b. Le istruzioni `test_and_set()` e `compare_and_swap()` non sono interrompibili
 - c. Il semaforo binario permette l'accesso esclusivo ad una risorsa condivisa
 - d. Il mutex generalizza il concetto di semaforo contatore